CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Code (Core Segments)

InterviewSign is written in TypeScript with React Native and Expo, and uses Supabase for authentication, the PostgreSQL database and file storage. The core segments of each module are given below as Code 4.1 to Code 4.8. Each segment is taken from the source file named in its caption.

Code 4.1: Login with email or username (login.tsx)

```
const handleLogin = async () => {
  setError("");
  setSubmitting(true);
  try {
    const value = identifier.trim().toLowerCase();
    let email = value;

    if (!value.includes("@")) {                      // a username was typed
      const { data, error: lookupError } = await supabase.rpc(
        "get_email_by_username", { p_username: value });
      if (lookupError || !data) {
        setError("Invalid username or password.");    // generic message
        return;
      }
      email = data as string;
    }

    const { error: signInError } = await supabase.auth.signInWithPassword({ email, password });
    if (signInError) setError("Invalid email/username or password.");
  } catch {
    setError("Something went wrong. Please try again.");
  } finally {
    setSubmitting(false);
  }
};
```

Code 4.2: Live username check with 500 ms delay (register.tsx)

```
useEffect(() => {
  if (username.length === 0) { setUsernameStatus("idle"); return; }
  if (!usernameRegex.test(username)) { setUsernameStatus("invalid"); return; }   // ^[a-z0-9_]{3,20}$

  setUsernameStatus("checking");
  const timer = setTimeout(async () => {
    const { data, error: rpcError } = await supabase.rpc(
      "is_username_available", { p_username: username });
    if (rpcError) { setUsernameStatus("error"); return; }
    setUsernameStatus(data ? "available" : "taken");
  }, 500);

  return () => clearTimeout(timer);          // cancel the old request on every key press
}, [username]);
```

Code 4.3: Validation and account creation with profile data (register.tsx)

```
const canSubmit =
  firstName.trim().length > 0 && lastName.trim().length > 0 &&
  usernameStatus === "available" && emailRegex.test(email.trim()) &&
  password.length >= 6 && confirmPassword.length > 0 && passwordsMatch && !submitting;

const { data, error: signUpError } = await supabase.auth.signUp({
  email: cleanEmail,
  password,
  options: { data: {                          // saved as user metadata, read by the database trigger
    first_name: firstName.trim(), last_name: lastName.trim(), username,
    phone: phone.trim(), city: city.trim(),
    hearing_status: hearingStatus, answer_preference: preference,
  } },
});
if (signUpError) { setError(signUpError.message); return; }
if (data.user && data.user.identities?.length === 0)
  setError("This email is already registered.");
```

Code 4.4: Round, option and topic selection (interview.tsx)

```
const selectRound = (id: string) => { setRoundId(id); setOptionId(null); setSelectedTopics([]); };

const toggleTopic = (item: string) =>
  setSelectedTopics((prev) =>
    prev.includes(item) ? prev.filter((t) => t !== item) : [...prev, item]);

const toggleAll = () => setSelectedTopics(allSelected ? [] : [...allTopics]);

const goBack = () => {
  if (step === "topic")  { setSelectedTopics([]); setStep("option"); }
  else if (step === "option") { setOptionId(null); setStep("round"); }
};

buttonEnabled = selectedTopics.length > 0;                // final step
buttonLabel = selectedTopics.length > 0
  ? `Start Interview (${selectedTopics.length} ${selectedTopics.length === 1 ? "topic" : "topics"})`
  : "Select at least one topic";
```

Code 4.5: History filter and score colour (history.tsx)

```
const getScoreColor = (score: number) => {
  if (score >= 75) return "#16a34a";     // green
  if (score >= 50) return "#d97706";     // amber
  return "#dc2626";                      // red
};

useEffect(() => {                        // category sent from a Home screen card
  if (category && filters.includes(category)) {
    setSelectedFilter(category);
    router.setParams({ category: undefined });
  }
}, [category]);

const filteredAttempts = selectedFilter === "All"
  ? attempts : attempts.filter((a) => a.category === selectedFilter);
```

Code 4.6: Profile photograph upload to Supabase Storage (profile.tsx)

```
const path = `${userData.id}/avatar`;                        // one folder per user
const { error: uploadError } = await supabase.storage.from("avatars")
  .upload(path, decode(asset.base64), { contentType: asset.mimeType ?? "image/jpeg", upsert: true });
if (uploadError) throw new Error(uploadError.message);

const { data } = supabase.storage.from("avatars").getPublicUrl(path);
const url = `${data.publicUrl}?t=${Date.now()}`;             // timestamp avoids the old cached photo

const { data: updated, error: updateError } = await supabase.from("profiles")
  .update({ avatar_url: url }).eq("id", userData.id).select();
if (updateError) throw new Error(updateError.message);
await refreshProfile();
```

Code 4.7: Protected navigation (_layout.tsx)

```
<Stack screenOptions={{ headerShown: false }}>
  <Stack.Protected guard={isAuthorized}>        {/* only logged-in users */}
    <Stack.Screen name="(tabs)" />
    <Stack.Screen name="edit-profile" />
  </Stack.Protected>
  <Stack.Protected guard={!isAuthorized}>       {/* only logged-out users */}
    <Stack.Screen name="login" />
    <Stack.Screen name="register" />
  </Stack.Protected>
</Stack>
```

Code 4.8: Profile trigger and 7-day username guard (Supabase SQL)

```
create trigger on_auth_user_created after insert on auth.users
  for each row execute function handle_new_user();

-- handle_new_user(): copies the sign-up metadata into the profiles table
insert into public.profiles (id, first_name, last_name, username, email,
                             phone, city, hearing_status, answer_preference)
values (new.id, new.raw_user_meta_data->>'first_name', new.raw_user_meta_data->>'last_name',
        lower(new.raw_user_meta_data->>'username'), new.email,
        nullif(new.raw_user_meta_data->>'phone', ''), nullif(new.raw_user_meta_data->>'city', ''),
        nullif(new.raw_user_meta_data->>'hearing_status', ''),
        coalesce(nullif(new.raw_user_meta_data->>'answer_preference', ''), 'Both'));

-- guard_profile_update(): BEFORE UPDATE on profiles
new.id := old.id;  new.email := old.email;  new.created_at := old.created_at;   -- locked fields
if lower(new.username) <> lower(old.username) then
  if old.username_changed_at is not null
     and now() < old.username_changed_at + interval '7 days' then
    raise exception 'USERNAME_COOLDOWN';
  end if;
  new.username_changed_at := now();
else
  new.username_changed_at := old.username_changed_at;
end if;
```

4.2 Testing Approach

Testing was done manually on a physical Android phone and an iOS simulator through Expo Go, connected to the live Supabase project. Two levels were used. Unit testing checks a single screen rule, validation or database function at a time. Integration testing checks complete flows in which the screens, the shared context and Supabase Authentication, Database and Storage work together. Every test case has an id, the input, the expected result and the result that was observed.

4.2.1 Unit Testing

Table 4.1 lists the unit test cases and their results.

Table 4.1: Unit Test Cases and Results

ID	Test Case	Input	Expected Result	Result
UT01	Login button state	Empty identifier, or password shorter than 6 characters	Log In button stays disabled	**Pass**
UT02	Login with username	Username without "@" and correct password	Email is fetched by RPC get_email_by_username and login succeeds	**Pass**
UT03	Wrong credentials	Valid email, wrong password	Generic message "Invalid email/username or password." is shown	**Pass**
UT04	Show / hide password	Tap the eye icon	Password is masked or shown and the icon changes	**Pass**
UT05	Username sanitising	Typing "Ali_Ansari#1"	Field shows "ali_ansari1"	**Pass**
UT06	Invalid username length	Username "ab"	Red message about 3-20 characters; no request is sent	**Pass**
UT07	Username available	Unused username "ali_ansari"	"Checking availability..." then, after 500 ms, green "Username is available."	**Pass**
UT08	Username taken	Username that already exists	Red message "Username is already taken."; Register disabled	**Pass**
UT09	Email validation	Email "user@@mail"	Register button stays disabled	**Pass**
UT10	Password confirmation	Password and confirm password differ	"Passwords do not match." is shown; Register disabled	**Pass**
UT11	Hearing status chip	Tap "Deaf" twice	First tap selects it, second tap clears the selection	**Pass**
UT12	Register enabled	All required fields valid	Register button becomes active	**Pass**
UT13	Round change	Select another round after choosing an option	Earlier option and topics are cleared	**Pass**
UT14	All Topics toggle	Tap "All Topics", then tap it again	All topics selected, then all cleared	**Pass**
UT15	Start button label	0 topics, then 3 topics selected	"Select at least one topic" (disabled), then "Start Interview (3 topics)"	**Pass**
UT16	Score colour	Scores 82, 75, 58, 50, 45	Green, green, amber, amber, red	**Pass**
UT17	History filter	Select chip "Technical"; select a category with no attempts	Only Technical attempts shown; "No attempts found" for an empty category	**Pass**
UT18	Profile display	User with no photo and no phone	Initials shown in the avatar and "Not added" for phone	**Pass**
UT19	Username cooldown (database)	Change username twice within 7 days	Second update raises USERNAME_COOLDOWN	**Pass**
UT20	Protected profile fields (database)	Update that tries to change email or created_at	Old values are kept by guard_profile_update()	**Pass**

4.2.2 Integration Testing

Table 4.2 lists the integration test cases. They follow the main paths of a user through the application and also test the security rules of the database.

Table 4.2: Integration Test Cases and Results

ID	Scenario	Steps / Modules Involved	Expected Result	Result
IT01	New user registration	Register form, Supabase Auth signUp, trigger handle_new_user, profiles table	Account is created, a profiles row exists with the entered details and the user lands on the Home screen	**Pass**
IT02	Duplicate email	Register with an email that is already registered	"This email is already registered." is shown	**Pass**
IT03	Duplicate username in database	Save a username that exists (unique index on lower(username))	Error 23505 is handled and a message is shown; no duplicate row is stored	**Pass**
IT04	Login with email	Login screen, Supabase Auth, AppProvider, profiles table	Session starts, profile is loaded into context and the Home screen shows the user name	**Pass**
IT05	Login with username	RPC get_email_by_username, then signInWithPassword	User is logged in exactly as with an email	**Pass**
IT06	Protected routes	Stack.Protected guards with a logged-out and a logged-in user	Logged-out users see only login and register; logged-in users see only the tabs and edit-profile	**Pass**
IT07	Logout	Profile screen, logout(), Supabase session	Session is cleared and the user returns to the Login screen	**Pass**
IT08	Profile photograph upload	Image picker, avatars bucket, profiles.avatar_url, refreshProfile()	Photo is stored at avatars/{userId}/avatar and appears in the profile header	**Pass**
IT09	Edit profile and username rule	Edit Profile screen, live availability check, guard trigger	Changes are saved; a second username change within 7 days is blocked	**Pass**
IT10	Interview selection flow	Interview tab: round, option, topics, Back button	Steps open in order, Back clears the later choices, and the start button needs at least one topic	**Pass**
IT11	Attempt saved and shown	Answer by text or sign video, feedback and score, interview_attempts, Home and History	Attempt is saved for the user and appears in History, in Recent Attempts and in the statistics	**Pass**
IT12	Home to History category	Tap the "Technical" card on Home	History opens with the Technical chip selected and scrolled to the centre	**Pass**
IT13	Row Level Security	Log in as user B and query the rows and files of user A	Zero rows are returned and files of user A cannot be read or changed	**Pass**
IT14	Question bank access	Read rounds, round_options, topics and questions with and without login	Logged-in users can read them; logged-out users cannot	**Pass**

Test summary. In total 34 test cases were run: 20 unit tests and 14 integration tests. All of them passed. No failed case remained open after the final run.

Table 4.3: Test Summary

Level	Test Cases	Passed	Failed	Pass Rate
Unit Testing	20	20	0	100%
Integration Testing	14	14	0	100%
Total	**34**	**34**	**0**	**100%**